START / 1분 소개

먼저 외울 것은 코드가 아니라 내 역할

저는 사용자와 대화 데이터를 저장하고, AI와 화면에서 필요한 기록을 안전하게 조회할 수 있도록 DB·로그 계층을 구현했습니다.

평가 시작 60초 답변

> DB 담당 C를 맡았습니다. Neon PostgreSQL의 개발·운영 환경을 나누고 SQLAlchemy 엔진과 동기 Session 기반을 구성했습니다. 사용자 조회·생성과 ChatLog 모델, 대화 저장·최근 문맥 조회·본인 기록 조회 함수를 만들었습니다. 조회 API는 공통 인증을 재사용하고, DB 저장 오류는 rollback 후 전달하도록 했습니다. 또 테이블 생성 스크립트와 확인 SQL, 테스트, ERD와 DB 문서를 작성했습니다. AI 호출은 B, 인증은 A, 배포와 화면 연결은 D가 담당했습니다.

이 자료를 공부하는 순서

회차	읽을 부분	목표
1회 · 20분	1-4쪽, 17쪽	역할·용어·전체 흐름을 말한다
2회 · 40분	5-14쪽	코드를 보며 이유까지 설명한다
3회 · 30분	15-16쪽, 17쪽	답을 가리고 구술 문제를 푼다

> **공부 기준**
>
> 각 기능은 '무엇을 하는가 → 왜 필요한가 → 어느 코드인가 → 어떻게 검증했는가'의 네 문장으로 설명한다. 문법을 통째로 외우기보다 입력·처리·반환·실패를 먼저 잡는다.

이 자료는 실제 저장소와 본인 커밋에 근거한 개인 학습용 문서다. 과제 원문이 로컬에서 발견되지 않아 평가 항목별 공식 합격 판정은 하지 않는다. 가상의 예시 데이터와 축약 코드는 학습용임을 별도로 표시했다.

서비스: https://b7-ai-chatbot.vercel.app/ · 코드: Teamb7-1/ai-chatbot-service. 앱 코드는 수정하지 않고, Git 저장소 밖에 자료를 만들었다.

OWNERSHIP / 기여 근거

내가 한 것과 팀원이 한 것

작성자 Geonyoung Choi / 00skgun@naver.com 기준, 병합 커밋을 제외한 기능·테스트·문서 커밋 11개가 확인됐다. 숫자를 늘린 것이 아니라 세 가지 작업 묶음으로 나누어 구현했다.

이슈 / PR	내 작업	커밋
#70 / PR #77	DB 패키지, engine·Base·Session, 계약 테스트	3개
#66 / PR #85	사용자 조회·생성, 실패 시 rollback, PostgreSQL 검증	2개
#36 / PR #90	ChatLog·대화 CRUD·조회 API·초기화·SQL·테스트·문서	6개

역할 경계 - 평가에서 과장하지 않기

담당	구현 책임	C와 만나는 지점
A · yun-lim	User 모델, 비밀번호/JWT, 인증·Session 의존성	C의 사용자 CRUD와 DB 기반을 사용
B · sonjehyun123-maker	AI 클라이언트, 문맥 조립, 챗 파이프라인	C의 recent_turns·create_chat_log를 호출
C · 00skgun	DB 기반, 사용자·대화 CRUD, ChatLog, 기록 API	데이터 저장·조회 계약을 제공
D · zxcv718	앱·화면·공통 응답·로깅·배포·운영 DB 적용	C 라우터 등록, 화면 조회 연결, 스크립트 실행

정확한 표현

'회원가입·로그인을 전부 만들었다'가 아니라 '인증에서 사용하는 사용자 조회·생성 DB 함수를 만들었다.' '회원가입·로그인을 전부 만들었다'가 아니라 '인증에서 사용하는 사용자 조회·생성 DB 함수를 만들었다.' 'Neon 테이블 생성 스크립트를 만들었다'와 '실제 두 브랜치에 실행했다(D)'도 구분한다.

추가 기여: Neon 제공자 선정, development·production 브랜치 분리, pooled 연결 정보 비공개 전달. Git 밖에서 한 환경 준비는 커밋 수와 별개다.

```
git log --no-merges --author=00skgun@naver.com --oneline origin/develop
```

PR 작성자가 D로 표시되는 이유는 자동 PR이 D의 토큰으로 생성되기 때문이다. 실제 코드 작성자는 커밋 author와 PR 본문의 작업자 항목으로 확인한다. 최신 팀 코드에는 B의 파이프라인과 D의 라우터·화면 연결도 반영돼 있다.

BASICS / 용어 정리

헷갈리는 이름부터 분리하기

이름	쉽게 말하면	우리 프로젝트에서
PostgreSQL	관계형 DB를 실제로 운영하는 소프트웨어	사용자·대화 테이블의 데이터 저장
Neon	PostgreSQL을 관리해서 제공하는 서비스	외부 DB 서버 제공자
SQLAlchemy	Python 객체와 SQL/DB 작업을 연결하는 라이브러리	모델·쿼리·Session·engine
psycopg	Python이 PostgreSQL과 통신하는 드라이버	SQLAlchemy 아래에서 DB 통신
ORM	테이블 행을 Python 객체로 다루는 방식	ChatLog 객체의 .question 사용
Pydantic	API 입력·출력 구조를 검증하는 도구	ChatLogItem 응답 형식
SQL	DB에 조회·저장을 요청하는 언어	SELECT, INSERT 등
CRUD	Create·Read·Update·Delete의 약자	이번 범위는 주로 생성과 조회

왜 SQLite 대신 Neon PostgreSQL인가?

SQLite는 파일 기반이라 로컬 실습에는 간단하다. 하지만 팀은 Vercel 서버리스 배포를 선택했고 함수의 임시 파일을 영구 저장소로 삼을 수 없다. 함수 밖의 관리형 PostgreSQL을 사용해 애플리케이션 재배포와 데이터 저장을 분리했다. SQLite 자체가 나쁜 DB라는 뜻은 아니다.

Neon은 pooled 연결을 제공하고, 유휴 상태의 컴퓨트는 연결 요청에 따라 다시 기동한다. 개발용·운영용 DB를 브랜치로 분리할 수 있어 테스트 데이터를 운영 데이터와 나누는 데 사용했다. 무료 한도는 바뀔 수 있으므로 가격 숫자보다 선정 이유를 설명한다. [N1, N2]

```
postgresql+psycopg://USER:PASSWORD@HOST/DB?sslmode=require
# 학습용 형식. 실제 연결 정보가 아니다.
```

비밀값 주의

DATABASE_URL은 환경변수 이름(Key), 위 형식의 접속 문자열은 값(Value)이다. Neon 콘솔 웹주소는 DB 접속 문자열이 아니다. 실제 비밀번호·URL·쿠키는 README·이슈·캡처에 공개하지 않는다.

Neon 브랜치는 데이터의 분기이고 Git 브랜치는 코드의 분기다. development에 테이블을 만들어도 production에 자동으로 생기지는 않으므로 초기화는 두 환경에 각각 수행한다.

FLOW / 입력부터 저장까지

질문 한 번이 내 코드를 지나는 과정

01. 사용자가 질문 전송 → D의 main.py가 request_id를 붙이고 B의 POST /api/chat으로 보낸다.

02. A의 CurrentUser가 쿠키를 검증하고 현재 User를 구한다. DbSession이 요청의 DB Session을 제공한다.

03. B의 chat_service가 C의 recent_turns(db, user.id, n=5)를 호출한다.

04. C는 본인의 최근 성공 기록을 조회해 과거 → 현재 순서의 ChatLog 객체 목록을 반환한다.

05. B는 .question/.answer를 읽어 messages를 조립하고 새 질문을 붙여 AI를 호출한다.

06. AI 결과가 오면 B가 C의 create_chat_log를 호출한다. C는 질문·답변·결과 정보를 저장하고 commit한다.

07. B는 저장된 log.id와 답변으로 ChatResponse를 반환한다. D의 화면이 답변을 표시한다.

08. 지난 대화에서는 D의 /logs 화면 또는 C의 GET /api/me/chats가 C의 list_chat_logs를 사용한다.

> **핵심 문장**
>
> 저는 AI를 직접 호출한 것이 아니라, AI에 줄 과거 대화를 조회하고 새 결과를 저장하는 데이터 계층을 만들었습니다. 같은 CRUD 함수를 서비스·API·화면이 재사용합니다.

최근 5턴은 무엇인가?

우리 ChatLog 한 행은 질문 1개와 그 결과 1개를 묶은 한 턴이다. 성공 5행이면 과거 user/assistant 메시지 최대 10개가 되고, B가 현재 질문 1개를 추가한다. 5개의 개별 메시지만 가져오는 것이 아니다.

코드 표현	뜻
db: Session	db 인자로 SQLAlchemy Session을 받는다는 타입 표시
n: int = 5	n은 정수이고 생략하면 기본 5를 사용
-> list[ChatLog]	ChatLog 객체들의 목록을 반환한다는 표시
def / async def	동기 함수 / 코루틴 함수. C의 CRUD는 def
raise	오류를 호출자에게 전달. 조용히 성공으로 바꾸지 않음

팀의 공통 계약이 동기 SQLAlchemy Session이어서 C의 CRUD도 일반 def로 작성했다. 이를 B의 async 함수에서 호출한다고 DB 작업까지 자동 비동기가 되지는 않는다. 규모가 커지면 비동기 DB 드라이버·Session 전환 등을 함께 검토할 수 있다. '비동기가 항상 빠르다'고 답하지 않는다.

CODE 01 / #70

database.py - DB에 접근하는 공통 기반

```
engine = create_engine(_database_url(), pool_pre_ping=True)

class Base(DeclarativeBase):
    pass  # 설명용 축약

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)
```

구성	정확한 역할	선택 이유
_database_url()	환경변수 확인 후 앞뒤 공백을 제거해 반환	없으면 명확히 실패. 주소를 코드에 고정하지 않음
engine	드라이버·연결 풀을 관리하는 DB 접근 시작점	여러 함수가 공통 연결 설정을 사용
Base	ORM 모델의 공통 부모와 metadata 제공	User·ChatLog의 테이블 정의를 모음
SessionLocal	Session 객체를 만드는 팩토리	요청마다 필요한 Session을 생성
pool_pre_ping=True	풀에서 연결을 꺼낼 때 유효성 확인	끊어진 연결을 재사용할 위험을 줄임
autoflush=False	조회 등에 앞선 자동 flush를 끔	저장 시점을 코드로 명시
expire_on_commit=False	commit 뒤 객체 속성의 자동 만료를 막음	저장 결과를 사용할 때 불필요한 재조회를 줄임

engine 생성 ≠ 연결 검증 ≠ 테이블 생성

create_engine()은 보통 첫 실제 DB 작업까지 연결하지 않는다. import·/healthz가 성공해도 DB 접속과 테이블 존재가 검증된 것은 아니다. 이 때문에 별도의 초기화·실제 요청 검증이 필요하다. [S1]

Session은 ORM 객체와 트랜잭션을 관리하는 작업 단위다. 사용자 로그인 Session과 다른 뜻이다. A의 get_db()가 SessionLocal()로 만들고 요청 종료 시 finally에서 close()한다. C CRUD는 전달받은 db를 사용하고 직접 닫지 않는다.

autoflush=False여도 commit은 남은 변경을 flush할 수 있다. pool_pre_ping은 실행 중인 트랜잭션 실패를 자동 복구하거나 재시도하는 옵션이 아니다. Neon의 서버 측 풀러와 SQLAlchemy의 앱 측 풀은 위치가 다르다. [S1, S2, S4, N1]

CODE 02 / ChatLog 11개 컬럼

models.py - 대화 한 건에 무엇을 저장했나

User 모델은 A의 구현이다. C는 users.id를 참조하는 ChatLog 모델을 추가했다. 한 사용자는 여러 대화를 가질 수 있고, 대화 한 행은 사용자 한 명에 속한다.

컬럼	타입 / 제약	저장 이유
id	BIGINT, 자동 증가 PK	대화 행 식별
user_id	BIGINT, users.id FK	누구의 기록인지 구분
request_id	TEXT, 인덱스	서버 요청 로그와 연결
question	TEXT, NOT NULL	질문 원문
answer	TEXT, NOT NULL	답변 원문. AI 실패는 빈 문자열
status	TEXT, success 또는 error	성공·실패 구분
error_code	TEXT, NULL 허용	실패 원인. 성공 시 None
provider	TEXT	AI 제공자
model	TEXT	사용한 AI 모델
latency_ms	INTEGER, 0 이상	B가 계산해 전달한 AI 경과 시간 (ms)
created_at	TIMESTAMPTZ, DB now()	기록 시각

PK는 행의 고유 식별자다. FK는 존재하는 사용자만 참조하도록 한다. FK 자체가 '남의 기록 조회'를 막는 것은 아니다. 그 보안 조건은 인증된 user.id와 WHERE user_id 필터가 담당한다.

인덱스: (user_id, created_at, id)는 사용자별 시간순 조회를 돕는다. request_id에도 인덱스가 있지만 고유 제약은 아니다. 인덱스는 조회를 도울 수 있는 대신 공간과 쓰기 비용이 든다.

```
@property
def chat_id(self) -> int:
    return self.id
```

이름이 달라도 같은 값

B는 log.id를 쓰고 API의 ChatLogItem은 chat_id를 요구한다. @property로 같은 값을 제공했다. chat_id는 12번째 DB 컬럼이 아니다. TIMESTAMPTZ는 시간대를 고려한 시각이며 원래 지역명까지 저장하는 것은 아니다. now()는 트랜잭션 시작 시각이라 동률을 대비해 id로도 정렬한다. [P1, P2]

CODE 03 / #66

사용자 CRUD - 인증에 필요한 DB 함수

함수	입력 → 결과	쓰이는 곳
get_user_by_username	db, username → User 또는 None	가입 중복 확인·로그인 조회
get_user_by_id	db, user_id → User 또는 None	인증 토큰의 사용자 ID로 실제 사용자 확인
create_user	db, username, password_hash → User	회원가입 데이터 저장

```
return db.scalar(
    select(User).where(User.username == username)
)

return db.get(User, user_id)
```

select(User)는 User를 조회하는 문장을 만든다. where는 조건을 붙인다. db.scalar는 첫 스칼라 결과, 여기서는 User 객체 하나를 받는다. db.get은 기본키 조회이며 같은 Session의 메모리에 객체가 있으면 그 객체를 사용할 수도 있다.

create_user가 하는 일

01. A의 인증 코드가 비밀번호를 해시한 다음 password_hash를 C 함수에 전달한다.

02. C가 User 객체를 만들고 add → flush → refresh → commit한다.

03. SQLAlchemyError가 발생하면 rollback 후 같은 예외를 다시 올린다.

04. 호출자가 중복 사용자 오류를 409 등 공통 API 오류로 해석한다. C CRUD가 HTTP 상태를 결정하지 않는다.

> ### 내가 하지 않은 것
>
> C는 비밀번호 평문을 해시하거나 JWT를 발급하지 않는다. User 테이블 모델과 로그인 인증도 A의 책임이다. 나는 이미 해시된 값을 저장하고 필요한 사용자를 조회하는 부분을 만들었다.

username 중복은 DB의 고유 제약이 최종적으로 막는다. 먼저 조회했어도 동시 가입 요청 사이에 중복이 생길 수 있으므로 DB 오류 처리가 필요하다. 모든 IntegrityError를 중복 사용자라고 처리하지 않고 호출자가 PostgreSQL의 오류 종류와 제약 이름을 구분한다.

ORM 조건에는 값이 파라미터로 바인딩된다. 문자열을 붙여 SQL을 만들지 않는다. 단, ORM을 쓴다고 모든 접근 제어 문제가 저절로 해결되는 것은 아니며 사용자 필터는 별도로 필요하다.

CODE 04 / create_chat_log

저장의 핵심 - commit과 rollback

create_chat_log는 user_id·request_id·질문·답변·성공 여부·오류 코드·제공자·모델·소요 시간을 받는다. id와 created_at은 DB에서 채운다. 결과는 저장된 ChatLog 객체다.

```
try:
    db.add(chat_log)
    db.flush()
    db.refresh(chat_log)
    db.commit()
except SQLAlchemyError:
    db.rollback()
    raise
# 설명용 핵심 발췌. 실제 코드에는 안전한 로그 기록이 있다.
```

순서	무슨 일인가	혼동하면 안 되는 점
add	Session에 새 객체를 등록	이 한 줄만으로 영구 저장된 것이 아님
flush	INSERT 등 SQL을 DB에 보내 변경 반영	아직 commit 전이므로 취소 가능
refresh	DB 값을 다시 읽어 객체를 갱신	id·created_at 등 DB 생성값 확인
commit	현재 트랜잭션을 확정	이후 rollback으로 이미 확정한 과거 작업을 취소하지 못함
rollback	실패한 현재 트랜잭션을 취소·Session 상태 복구	실패한 저장이 성공으로 바뀌는 것은 아님

refresh를 commit 전에 둔 이유는 재조회가 실패한 경우에도 저장 트랜잭션을 rollback할 수 있도록 하기 위해서다. flush 실패 후 같은 Session을 계속 쓰려면 명시적 rollback이 필요하다. commit은 새 객체 하나만이 아니라 현재 트랜잭션 전체를 확정하므로 무관한 변경을 같은 Session에 섞지 않는다. [S1, S2]

상황	기록 / 반환	결과 해석
AI 성공, DB 정상	status=success, answer 원문, error_code=None	대화 저장 성공
AI 실패, DB 정상	status=error, answer='', 오류 코드	실패한 AI 시도도 추적 가능
DB 저장 실패	rollback·db_save_failed 후 예외 전파	해당 대화가 DB에 남는다고 보장할 수 없음

보안·정직한 오류 처리

C는 성공 시 db_save_success, 실패 시 db_save_failed를 남긴다. 오류 로그에는 예외 종류만 기록하고 SQL 예외 본문에 들어갈 수 있는 질문·답변·접속 정보를 그대로 출력하지 않는다. 빈 성공 응답으로 실패를 숨기지 않는다.

CODE 05 / 가장 중요한 조회

recent_turns - 최근 성공 대화만 순서대로

```
statement = (
    select(ChatLog)
    .where(ChatLog.user_id == user_id,
           ChatLog.status == 'success')
    .order_by(ChatLog.created_at.desc(), ChatLog.id.desc())
    .limit(n)
)
return list(reversed(db.scalars(statement).all()))
```

01. WHERE: 본인 user_id와 success 조건을 먼저 적용한다.

02. ORDER BY: created_at 내림차순, 같은 시각이면 id 내림차순으로 정렬한다.

03. LIMIT: 그중 최근 n개만 가져온다. 기본값은 config.py의 AI_CONTEXT_TURNS=5다.

04. reversed: 뽑힌 n개를 뒤집어 B가 읽기 좋은 과거 → 현재 순서로 반환한다.

가상 기록: 오래된 순	사용자 / 상태	민수의 성공 문맥에 포함?
1	민수 / success	대상
2	민수 / error	실패라 제외
3	지수 / success	다른 사용자라 제외
4	민수 / success	대상
5	민수 / error	실패라 제외
6	민수 / success	대상

위 예시에서 민수의 n=2 결과는? (번호 순서가 시각 순서와 같다고 가정)

먼저 민수의 성공 기록만 남기면 1, 4, 6이다. 최신순 6, 4, 1에서 2개를 고르면 6, 4이고, 뒤집어 최종 반환은 [4, 6]이다.

평가 단골 함정

'최근 n개를 가져온 뒤 실패를 제거'하면 성공 기록 수가 n보다 적어질 수 있다. 우리 코드는 실패·남의 기록을 먼저 걸러낸 뒤 LIMIT한다. n=0은 [], n<0은 ValueError, 기록이 없으면 []다.

성공만 문맥에 넣는 이유: 실패한 AI 턴은 답변이 비어 있어 대화 문맥으로 부적절하기 때문이다. 보관 자체는 실패도 하지만 AI에 다시 보낼 문맥에서는 제외한다. 반환은 dict가 아니라 ChatLog 객체 목록이다.

CODE 06 / list_chat_logs · GET API

본인 기록 조회 - CRUD와 API의 경계

비교	recent_turns	list_chat_logs
사용처	B의 AI 문맥	기록 API·지난 대화 화면
사용자 조건	본인만	본인만
상태 조건	성공만	성공·실패 모두
정렬 결과	최근 n개를 과거 → 현재	최신 → 과거
개수·페이지	n, 기본 5	limit=20, offset=0
반환	list[ChatLog]	list[ChatLog]

limit는 몇 개를 가져올지, offset은 최신순 앞에서 몇 개를 건너뛸지다. limit=20, offset=20이면 앞의 20개를 건너뛴 다음 최대 20개를 본다. 페이지 사이에 새 기록이 생기면 offset 페이지 내용이 달라질 수 있어 무조건 고정된 스냅샷은 아니다.

```
def list_my_chats(user: CurrentUser, db: DbSession,
                  limit: Annotated[int, Query(ge=0)] = 20,
                  offset: Annotated[int, Query(ge=0)] = 0):
    logs = crud.list_chat_logs(db, user.id,
                               limit=limit, offset=offset)
    return [ChatLogItem.model_validate(log) for log in logs]
# 설명용 발췌. 실제 코드에는 SQLAlchemyError 처리도 있다.
```

CurrentUser는 A의 인증을 재사용해 얻은 User다. 요청자가 보내는 user_id를 믿지 않고 user.id로 조회한다. Query(ge=0)는 0 이상 정수를 요구한다. 현재 limit=0은 허용되고 별도의 최대 limit는 구현돼 있지 않다.

계층 / 표현	뜻
log.question	ORM 객체의 속성 접근
{'question': '질문'}	키와 값을 가진 Python dict
ChatLogItem.model_validate(log)	ORM 속성을 공통 응답 모델로 검증·변환
최종 JSON	FastAPI가 응답 모델을 JSON으로 직렬화

API는 chat_id, request_id, question, answer, status, error_code, latency_ms, created_at을 반환한다. password_hash·user_id·provider·model은 이 응답에 포함하지 않는다. 인증 실패는 401, 잘못된 페이지 인자는 422, 조회 DB 오류는 500 공통 오류다. DB 장애를 []로 위장하지 않는다.

사용자 격리의 두 단계

인증으로 '누구인가'를 확정하고, CRUD의 user_id 조건으로 '그 사람 데이터만' 조회한다. FK만으로는 이 격리가 되지 않는다. 응답 모델은 데이터 형식·노출 필드를 제한하는 또 다른 경계다.

OPERATIONS 01 / create_tables.py

테이블 생성은 앱 시작과 분리했다

Python에 ChatLog 클래스를 작성한 것과 실제 Neon에 chat_logs 테이블이 존재하는 것은 다르다. 모델은 설계도이고, 생성 스크립트는 그 설계대로 실제 테이블을 만드는 별도 작업이다.

```
Base.metadata.create_all(
    bind=bind,
    tables=[User.__table__, ChatLog.__table__],
    checkfirst=True,
)
```

01. D가 대상 Neon 브랜치와 DATABASE_URL 주입 상태를 확인한다.

02. 리포 루트에서 python -m scripts.create_tables --confirm을 실행한다.

03. 스크립트가 PostgreSQL인지 확인하고 users·chat_logs 중 없는 테이블을 만든다.

04. 성공하면 종료 코드 0, 설정·연결·권한 등 처리 대상 오류는 비밀값을 숨겨 보고하고 1로 종료한다.

05. D가 다른 브랜치의 연결 정보로 동일 절차를 수행한다. 두 브랜치를 자동으로 순회하는 스크립트는 아니다.

안전 장치	의미
--confirm 필요	실행 전 사람이 대상 환경을 확인하도록 함
checkfirst=True	기존 테이블을 다시 만들지 않음
대상 테이블 명시	User·ChatLog만 초기화 대상
.env 자동 로딩 안 함	실행 프로세스에 주입된 DATABASE_URL 사용
예외 본문 출력 안 함	접속 정보 등 비밀값 노출 위험 축소
finally: engine.dispose()	스크립트 종료 시 엔진 연결 자원 정리

생성 스크립트 ≠ 마이그레이션

이미 있는 테이블의 컬럼을 추가·수정·삭제하지 않는다. 모델 코드를 바꾸고 create_all을 다시 실행한다고 기존 DB 구조까지 바뀌지는 않는다. --confirm도 DB 브랜치를 자동 판별하는 기능은 아니므로 대상은 사람이 확인한다. [S3]

앱 시작 훅에서 실행하지 않는 이유: 서버리스 인스턴스의 기동마다 DDL을 실행하지 않고, 환경별 DB 변경을 명시적 작업으로 관리하기 위해서다. 이 프로젝트에서는 스크립트 작성이 C, 실제 Neon 실행이 D의 책임이다.

OPERATIONS 02 / 확인 SQL·재배포

저장됐다는 것을 어떻게 증명하나

check_logs.sql 순서	확인하는 것
1. DB·스키마·테이블	올바른 DB에 users·chat_logs가 있는가
2. 컬럼 조회	chat_logs가 11개 컬럼인가
3. 최근 20건	질문·답변·상태·request_id가 저장됐는가
4. 사용자·상태 집계	누구의 성공·실패 기록이 몇 건인가
5. 최근 성공 5건	recent_turns와 같은 조건·순서로 조회되는가
6. 특정 chat_id	재배포 전 기록이 재배포 후에도 그대로 있는가

```
-- 학습용 읽기 전용 예시: 123은 실제 확인할 ID로 바꾼다.
SELECT id, request_id, question, answer, created_at
FROM chat_logs
WHERE id = 123;
```

스크립트의 5·6번은 user_id=0, id=0 자리표시자다. 시연 계정과 대상 대화의 실제 ID로 바꿔야 한다. 이를 그대로 실행해 0건이 나온다고 저장 실패로 판단하지 않는다.

재배포 보존 확인 순서

01. 테스트용 가상 질문을 한 번 보내고 답변을 받은 뒤, chat_id·request_id·내용·시각을 기록한다.

02. D가 같은 배포 환경을 재배포한다. development와 production을 서로 비교하지 않는다.

03. 같은 Neon 브랜치에서 같은 chat_id를 조회한다. 기존 내용·시각이 유지되는지 확인한다.

04. 다른 계정에서는 그 기록이 보이지 않는지 확인한다. 캡처에는 비밀번호·쿠키·개인정보를 포함하지 않는다.

> **관측된 사실과 설계 이유를 구분**
>
> 외부 DB라서 데이터가 앱 파일과 분리되는 것은 설계 이유다. 특정 배포 뒤 데이터가 실제 남았다는 것은 전후 조회 결과로 증명한다. '외부 DB니까 검사 없이 무조건 통과'라고 답하지 않는다.

D는 #89 댓글에서 두 브랜치 테이블 생성, 스테이징의 대화 2건 저장·문맥 유지·SQL 조회, 이후 배포 뒤 동일 chat_id 보존을 확인했다고 보고했다. production 전체 흐름 확인은 팀 공유 메시지의 보고이며, C가 직접 그 운영 검증을 수행했다고 서술하지 않는다.

psql은 SQLAlchemy 전용 postgresql+psycopg:// 형식을 그대로 쓰지 않는다. 실제 실행은 D의 안전한 접속 설정을 사용한다. 평가 중에는 데이터 삭제·초기화가 아니라 조회로 증명한다.

EVIDENCE / 근거를 말하는 방법

테스트 - 무엇을 확인했고 무엇은 안 했나

테스트	대표 검증
test_database.py	Base·동기 Session 옵션, 환경변수 누락 실패
test_crud.py / test_crud_postgresql.py	사용자 조회·생성·중복·실패 후 rollback
test_chat_models.py	11개 컬럼·FK·제약·chat_id 응답 변환
test_chat_crud.py	저장 순서·오류 전파·필터·정렬·기본값
test_chat_crud_postgresql.py	실제 저장·조회·외래키/상태/지연시간 제약·세션 재사용
test_logs.py	인증·본인 기록·페이지 인자·응답 필드·DB 오류
test_create_tables.py	수동 확인·오류 종료·반복 실행 보존·SQL 실행

Mock 테스트는 DB 객체를 대역으로 바꿔 함수 호출·순서·예외 처리를 빠르게 검사한다. PostgreSQL 테스트는 실제 DB에서 자료형·제약·트랜잭션·정렬·저장을 확인한다. Mock 통과만으로 실제 PostgreSQL 동작을 단정하지 않는다.

운영 DB를 건드리지 않는 테스트 구조

01. TEST_DATABASE_URL을 명시한 별도 테스트 DB만 사용한다. 앱의 DATABASE_URL로 대신 접속하지 않는다.

02. 테스트마다 test_crud_...라는 고유 스키마와 외부 트랜잭션을 만든다.

03. Session은 savepoint 방식으로 결합한다. CRUD의 commit을 검사하더라도 최종 외부 트랜잭션 rollback으로 테스트 테이블·데이터를 정리한다.

04. TEST_DATABASE_URL이 없으면 실제 DB 테스트만 이유와 함께 skip한다.

168개 통과를 정확하게 말하기

2026-09-12 당시 전체 프로젝트 테스트 168개가 임시 PostgreSQL을 포함해 통과했다. C 혼자 168개 테스트를 전부 작성했다는 뜻은 아니다. 같은 시점 구성에서 실제 DB 없이 재실행한 결과는 151 passed, 17 skipped였다. 최신 코드의 현재 테스트 수와도 구분한다.

C의 조회 API 테스트는 별도 테스트 앱에 라우터를 등록한다. API 자체는 검증하지만 운영 앱 main.py의 등록까지 증명하지는 않는다. 그 연결은 D가 구현했고 실제 앱·배포에서 추가 확인했다.

평가 답변: '단위 테스트와 실제 PostgreSQL 테스트를 나눴고, 사용자 격리·정렬·실패 후 재사용·반복 초기화를 확인했습니다. 실제 배포 연결과 재배포 보존 결과는 D가 검증해 #89에 공유했습니다.'

REASONING / 트러블슈팅과 한계

왜 이렇게 했는지 설명할 두 가지 사례

사례 1. 저장 실패 후에도 Session을 안전하게 쓰려면?

문제: 중복 사용자나 잘못된 FK 등으로 flush가 실패하면 같은 Session을 바로 계속 사용할 수 없다. 대응: SQLAlchemyError를 잡아 rollback하고 원래 예외를 전달했다. 검증: 실패를 발생시킨 다음 같은 Session으로 올바른 데이터를 저장하는 PostgreSQL 테스트를 작성했다. 운영 장애를 직접 겪었다고 과장하지 않고 실패 시나리오를 재현·검증했다고 말한다.

사례 2. B는 id, D 응답 모델은 chat_id를 쓰는데?

문제: ChatLog DB 기본키는 id인데 공통 API 응답은 chat_id다. 대응: @property chat_id가 self.id를 반환하게 해 두 계약을 연결했다. 검증: ChatLogItem.model_validate(log) 결과의 chat_id가 log.id와 같은지 테스트했다. 다른 팀원의 스키마를 바꾸거나 중복 DB 컬럼을 만들지 않았다.

절대 이렇게 답하지 않기

잘못된 답	정확한 답
rollback하면 과거 데이터도 다 지워진다	현재 미확정 트랜잭션을 취소한다
실패 기록은 무조건 저장된다	DB 저장이 가능할 때 AI 실패도 기록한다
FK가 남의 기록 조회를 막는다	인증 user.id와 WHERE 조건으로 격리한다
async 함수에서 호출하면 CRUD도 비동기다	C는 동기이며 자동 전환되지 않는다
create_all이 바뀐 컬럼도 고쳐 준다	기존 스키마 수정은 마이그레이션 영역이다
제가 로그인·AI·배포까지 전부 구현했다	C의 DB 기능과 A·B·D의 연결을 구분한다

질문이 개선 방향으로 넘어오면

현재 계약을 먼저 설명하고, 대규모 트래픽에는 페이지 크기 상한·동시성 설계, 스키마 변경에는 마이그레이션, 운영 검증에는 자동 E2E 등을 검토할 수 있다고 답한다. 아직 없는 기능을 이미 구현했다고 말하지 않는다.

RECALL / 답을 가리고 연습

예상 질문 1-9 - 역할과 데이터 기반

1. 본인 담당을 한 문장으로 설명해 보세요.

사용자와 대화 데이터를 저장하고, AI·화면이 필요한 기록을 본인 범위에서 조회하도록 DB·로그 계층을 구현했습니다.

2. PostgreSQL과 Neon은 무엇이 다른가요?

PostgreSQL은 DB 소프트웨어이고 Neon은 그 PostgreSQL을 관리형 서비스로 제공하는 제공자입니다.

3. 왜 SQLite를 사용하지 않았나요?

팀의 Vercel 서버리스 배포에서는 로컬 임시 파일의 영속성에 의존하지 않기 위해 외부 PostgreSQL을 선택했습니다. SQLite 자체가 부적합한 DB라는 뜻은 아닙니다.

4. SQLAlchemy와 psycopg의 차이는?

SQLAlchemy는 ORM·쿼리·Session·연결 풀을 제공하고 psycopg는 Python과 PostgreSQL 사이의 통신 드라이버입니다.

5. engine과 Session은 같은 건가요?

아닙니다. engine은 드라이버와 연결 풀의 공통 접근 기반이고, Session은 요청의 ORM 객체와 트랜잭션을 관리하는 작업 단위입니다.

6. autoflush=False이면 저장이 안 되는 것 아닌가요?

조회 등에 앞선 자동 flush만 끕니다. 저장 코드에서 flush를 명시하고, commit 때도 필요한 flush가 수행될 수 있습니다.

7. expire_on_commit=False는 왜 넣었나요?

commit 후 객체 속성을 자동 만료시키지 않아 저장 결과의 id나 답변 등을 읽을 때 불필요한 재조회가 발생하는 것을 줄입니다.

8. ChatLog에는 왜 11개 컬럼이 있나요?

질문·답변·사용자·시간에 더해 행 ID, request_id, 상태·오류 코드, 제공자·모델, 지연 시간을 저장해 결과와 실패 원인을 추적하려고 구성했습니다.

9. User 모델과 회원가입 기능도 본인이 만들었나요?

User 모델·해싱·인증 API는 A의 작업입니다. 저는 사용자 조회·생성 DB 함수와 DB 실패 처리, 테스트를 만들었습니다.

RECALL / 이유까지 말하기

예상 질문 10-18 - 조회·오류·검증

10. flush와 commit의 차이는 무엇인가요?

flush는 SQL 변경을 DB에 보내지만 트랜잭션은 아직 확정 전입니다. commit은 현재 트랜잭션을 확정합니다. flush 후 commit 전 변경은 rollback할 수 있습니다.

11. refresh를 commit 전에 한 이유는?

DB에서 생성한 값을 다시 읽고, 그 조회까지 실패한 경우에도 아직 미확정인 저장 트랜잭션을 rollback할 수 있도록 했습니다.

12. 최근 성공 5개를 어떻게 가져오나요?

본인 user_id와 success를 먼저 필터하고 created_at·id 내림차순으로 5개를 선택한 뒤 목록을 뒤집습니다. B는 과거부터 현재까지 읽습니다.

13. 로그 화면도 성공한 대화만 보여 주나요?

아닙니다. recent_turns만 성공 조건을 쓰고 list_chat_logs는 실패도 포함합니다. 실패한 대화 역시 추적할 필요가 있기 때문입니다.

14. 다른 사람의 user_id를 요청에 넣으면 어떻게 되나요?

조회 API는 요청의 user_id를 쓰지 않고 인증된 CurrentUser의 user.id를 CRUD에 전달합니다. CRUD도 해당 user_id 조건으로 조회합니다.

15. ORM 객체를 왜 그대로 JSON 응답으로 주지 않나요?

DB 구조와 외부 응답 구조를 분리하기 위해서입니다. 공통 ChatLogItem으로 검증·변환해 약속된 필드만 노출하고 FastAPI가 JSON으로 직렬화합니다.

16. AI 실패와 DB 실패는 어떻게 다른가요?

AI가 실패해도 DB가 정상이면 status=error와 오류 코드를 저장합니다. DB 저장 자체가 실패하면 rollback하고 예외를 전달하므로 해당 기록의 저장은 보장할 수 없습니다.

17. 초기화 명령을 다시 실행하면 기존 데이터가 지워지나요?

아닙니다. checkfirst=True로 없는 테이블만 만듭니다. 단 기존 테이블 구조를 새 모델에 맞게 수정하는 마이그레이션 기능도 아닙니다.

18. 기능이 됐다는 근거는 무엇인가요?

사용자 격리·정렬·rollback·초기화 보존을 단위/실제 PostgreSQL 테스트로 확인했습니다. 9월 12일 전체 테스트 168개가 통과했고, D는 실제 환경의 SQL 조회와 재배포 보존 결과를 #89에 공유했습니다. 테스트 수와 운영 검증 주체는 구분합니다.

LAST CHECK / 한 장 복습

평가 10분 전 - 이 페이지만 다시 보기

나는 저장·조회·검증을 맡았다. 인증은 A, AI는 B, 화면·배포는 D다.

암기어	말로 풀기
Neon / PostgreSQL	제공자 / DB 소프트웨어
engine / SessionLocal / Session	접근 기반 / 팩토리 / 요청의 작업 단위
Base / ChatLog	공통 ORM 기반 / 대화 한 턴의 모델
add → flush → refresh → commit	등록 → SQL 반영 → DB값 재조회 → 확정
rollback → raise	현재 실패 작업 취소·복구 → 오류 전달
recent_turns	본인·성공 필터 → 최신 n개 → 뒤집기
list_chat_logs	본인·모든 상태 → 최신순 → limit/offset
CurrentUser + WHERE	인증된 사람 확정 + 데이터 소유권 제한
ChatLogItem	ORM → 약속된 API 응답 형식
create_all(checkfirst=True)	없는 테이블 생성. 데이터 삭제·스키마 수정 아님
증거	11개 커밋 / PR 77·85·90 / 테스트 / SQL·#89

마지막 구술 체크

- 코드를 안 보고 질문이 들어와 DB에 저장될 때까지의 흐름을 1분 안에 설명할 수 있다.
- recent_turns 가상 기록에서 [4, 6]이 나오는 이유를 직접 계산할 수 있다.
- 실패 시 rollback과 raise가 둘 다 필요한 이유를 말할 수 있다.
- 본인의 코드와 팀원의 연결·운영 검증을 구분할 수 있다.
- '168개를 내가 전부 작성했다', 'create_all은 migration이다' 같은 오답을 피할 수 있다.

> **막히면 이 순서로 답하기**
>
> 입력은 무엇인지 → 내 함수가 어떤 조건·순서로 처리하는지 → 반환값은 무엇인지 → 실패 시 무엇을 하는지 → 어떤 테스트로 확인했는지. 기억이 안 나는 세부 문법은 추측하지 말고 해당 파일을 열어 근거를 보여 준다.

더 공부할 시간이 있다면: ① database.py와 crud.py를 소리 내어 해설 ② 18문항을 답 없이 재시도 ③ 다른 계정·실패·빈 기록 사례로 결과를 예측. 실습은 테스트 DB에서 하고 운영 DB를 임의로 수정하지 않는다.

REFERENCE / 출처·검증 범위

코드 위치와 근거 - 모르면 여기서 찾기

질문	열어 볼 파일 / 함수
연결·Session 옵션	app/database.py / engine, SessionLocal
11개 컬럼·FK·인덱스	app/models.py / ChatLog
저장·실패 취소	app/crud.py / create_user, create_chat_log
문맥·최신 기록	app/crud.py / recent_turns, list_chat_logs
조회 API의 본인 확인	app/routers/logs.py / list_my_chats
인증·세션 주입(A)	app/deps.py / get_current_user, get_db
응답 형식(D)	app/schemas.py / ChatLogItem, AppError
AI에 문맥 전달(B)	app/services/chat_service.py / handle_chat
페이지·라우터 등록(D)	app/routers/pages.py, app/main.py
초기화·확인	scripts/create_tables.py, scripts/check_logs.sql
DB 설계·검증 안내	docs/ERD.md, README §7·§8

GitHub 근거

DB 기반 PR #77
사용자 CRUD PR #85
대화 DB 계층 PR #90
D의 실제 환경 검증 보고 #89

기술 개념 확인용 공식 문서

S1 · SQLAlchemy Session Basics
S2 · SQLAlchemy Session API
S3 · SQLAlchemy metadata / create_all
S4 · SQLAlchemy 연결 풀·pre_ping
P1 · PostgreSQL 날짜·시간
P2 · PostgreSQL now() / 트랜잭션 시각
N1 · Neon connection pooling
N2 · Neon scale to zero

기준일 2026-09-15. C 구현은 b5771be까지의 커밋과 최신 origin/develop(23a312f)을 대조했다. 이 문서 작성 시 운영 DB 접속·테스트 재실행·배포를 하지 않았다. 과거 테스트 결과는 실행일을 붙여 기록했다. 공식 평가표의 점수·항목 충족은 이 문서만으로 인증하지 않는다.